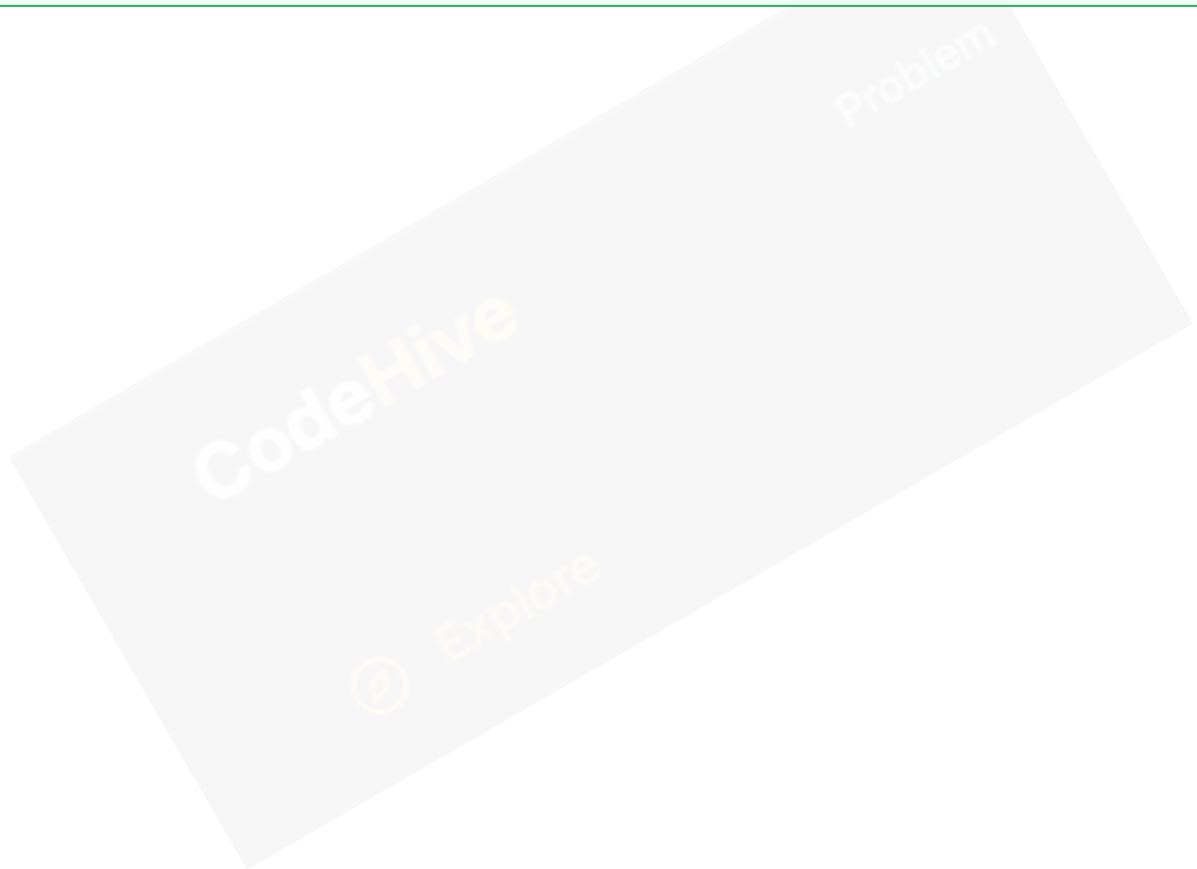


Enumerations (enum) in C

An **Enumeration** or **enum** is a user-defined data type used to assign names to integral constant values. It is primarily designed to make code more readable, meaningful, and self-documenting.

Visualisation: Instead of remembering that "0" means Monday and "1" means Tuesday, enums allow you to use the words MON and TUE directly in your logic.



1. Syntax and Basic Definition

Enums are defined using the enum keyword followed by a list of identifiers in curly braces.

```
enum Week {  
    MON, TUE, WED, THU, FRI, SAT, SUN  
};
```

Internal Mechanics: By default, the compiler assigns integer values starting from 0. So, MON = 0, TUE = 1, etc.

2. Declaring Enum Variables

Once the enum type is defined, you can create variables of that type to hold one of the constant names.

```
enum Week today;  
today = WED;  
  
if (today == WED) {  
    printf("It is the middle of the week!");  
}
```

CodeHive

@ Explore

3. Assigning Custom Values

You don't have to follow the 0, 1, 2... sequence. You can specify exact integer values for any or all constants.

```
enum Level {  
    LOW = 1,  
    MEDIUM = 5,  
    HIGH = 10  
};
```

This is useful for mapping code to specific protocol statuses or hardware flags.

CodeHive

@ Explore

4. Automatic Incrementing

If you only assign a value to the first member, the subsequent members will automatically increment by 1.

```
enum State {  
    START = 10,  
    PROCESSING, // Becomes 11  
    END         // Becomes 12  
};
```

CodeHive

@ Explore

5. Enum with typedef

To make declarations even cleaner, use typedef to avoid repeating the enum keyword.

```
typedef enum {  
    RED, GREEN, BLUE  
} Color;  
  
Color c = RED;
```

CodeHive

@ Explore

6. Why Use Enums? ★

- **Readability:** Replaces "magic numbers" (like 0, 1, 2) with clear descriptors.
- **Scope:** Unlike `#define`, enums follow scope rules.
- **Type Safety:** Helps the developer ensure only expected values are assigned.
- **Maintenance:** Changing an underlying value in the definition updates it everywhere else.



7. Real-World Use: State Machines

Enums are the standard way to implement finite state machines (e.g., Traffic Lights, Parser States).

```
enum TrafficState { RED, YELLOW, GREEN };  
enum TrafficState current = RED;
```

It makes the control flow (`switch` statements) incredibly clear.

8. Enum vs. #define

Feature	Enum	#define
Scope	Block/Global	Always Global
Type	Integral Type	Textual Substitution
Debugging	Visible in Debugger	Not Visible

CodeHive

Ⓢ Explore

Problem

9. Under the Hood: Size

C compilers usually treat enum variables as int. Therefore, `sizeof(enum Week)` is typically 4 bytes.

```
printf("%zu", sizeof(enum Week)); // Typically 4
```

CodeHive

@ Explore

Problem

10. Common Pitfalls ✓

- **Duplicate Values:** Two names can have the same value (e.g., `OK=1`, `SUCCESS=1`), but two names in the same scope cannot be identical.
- **Read-Only:** You cannot change an enum name's value at runtime (e.g., `MON = 5;` is illegal).



Key Points (Exam Ready)

- **One-Line Definition:** An enum is a user-defined data type that assigns names to integer constants.
 - It helps avoid using meaningless "magic numbers".
 - Default values are integers starting from 0.
 - Can be combined with **typedef** for cleaner declaration.
-

Powering Knowledge - CodeHive Official